

FeaturedCollection Block

1. Overview

The **Featured Collection component** is a server-side content inclusion component that pulls external content into the page at render time.

2. Current AEM Implementation

- Authors configure the component by providing **Featured Collection URL** (mandatory) that resolves to external featured content.

Example:

```
/search/browse/promotions/{promocode}?selector=noheader,nofooter
```

- At runtime, the component uses **Server-Side Includes (SSI)** to fetch and render external content.

Flow:

1. The component reads:
 - Authored **featuredCollectionUrl**
 - Incoming request query parameters
2. The component constructs an SSI include:
 - Appends request query parameters to the authored URL
3. The server executes the SSI directive:
 - Makes an HTTP call to the external service
 - Fetches the response
4. The response (HTML) is:
 - Injected directly into the page
 - Rendered as part of the server response

The external service returns:

- Pre-rendered HTML
- Associated CSS references
- JavaScript bundles
- Fully composed UI

The component does not control rendering and simply outputs the response.

3. Why HTML Injection is Not Recommended

The current approach relies on **injecting external HTML (with CSS and JS)**, which introduces key challenges:

3.1 Performance Impact

- External CSS may block rendering → impacts **LCP**
- External JS may block execution → impacts **TBT**
- Increased risk of layout shifts and delayed rendering

3.2 Lack of Control

- Markup structure is controlled externally
- Styling is not fully governed by the site
- Difficult to enforce:
 - design consistency
 - accessibility standards
 - semantic structure

3.3 Tight Coupling

- Frontend depends on backend HTML structure
- Backend changes can break UI
- Limits flexibility for UI evolution

3.4 EDS Compatibility

- **SSI is not supported in EDS**
- Injecting HTML with embedded CSS/JS is **not aligned with EDS architecture**
This pattern is **not suitable for EDS** due to performance and architectural constraints.

4. Recommended EDS Approach – JSON + Block Decoration

Replace SSI-based HTML inclusion with:

- **Backend → Provides structured data (JSON)**
- **EDS Block → Handles rendering (HTML, CSS, JS)**

Authors continue to provide: Featured collection identifier or URL

The **featured-collection block** will:

- Read authored configuration from the page
- Read relevant query parameters from the URL

- Construct a request to the backend JSON API
- Fetch structured data
- Render and Apply
 - controlled CSS
 - minimal, optimized JavaScript
- The backend should expose a **JSON API** instead of returning HTML.

The response should include:

- collection metadata
- item list (promotions, categories, etc.)
- images and navigation links
- optional paging or filtering data

This ensures:

- Backend handles **business logic and data**
- EDS handles **presentation and interaction**

This approach ensures:

- No uncontrolled CSS/JS injection
- Reduced payload size (JSON vs HTML)
- Ability to:
 - lazy load content
 - defer non-critical logic
 - optimize rendering

Aligns with EDS best practices for **Core Web Vitals and performance**